

第四题第 4.1 问详细解答

Poisson 方程的有限体积空间离散与 OpenFOAM v14 求解器

1 题目与必要说明

控制方程为

$$\nabla \cdot \nabla \phi = \omega, \quad \text{即} \quad \nabla^2 \phi = \omega. \quad (1)$$

要求基于有限体积法写出空间离散形式，并采用 OpenFOAM 提供的空间离散格式开发数值求解器。

方程 (1) 是稳态 Poisson 方程，本身没有时间导数。因此题目中的“半离散形式”应理解为：连续区域上的偏微分方程只在空间上作有限体积离散，得到单元中心未知量满足的代数方程。严格地说，它不是“保留连续时间变量”的时间半离散方程。

还需指出：Poisson 方程必须配合边界条件才构成闭合问题。第 4.1 问未给出具体边界值，以下先写一般边界通量；随后给出规定值边界 $\phi = g$ (OpenFOAM 中的 `fixedValue`) 的离散。若全部边界均为 Neumann 条件，则还需满足整体相容条件并固定一个参考值，否则解只确定到一个加法常数。

本文以用户此前提供的两本教材为主线：Moukalled、Mangani 与 Darwish 第 8 章给出扩散项从控制体积分到非正交修正及矩阵装配的数学推导 [1]；Marić、Höpken 与 Mooney 的 *The OpenFOAM Technology Primer* 第 1.3 节给出同一离散在 OpenFOAM owner-neighbour 数据结构中的实现逻辑，第 3.3 节和第 9.3 节分别说明运行时离散格式与新方程的程序实现 [2]。最后仅用 OpenFOAM Foundation v14 的用户指南和源代码核对当前版本的类名及几何系数 [3, 4, 5, 6, 7]。

本解答采用的自洽推导链

全文严格沿用下面一条逻辑链，不把公式推导、矩阵和代码作为互不相关的三部分：

$$\begin{aligned} \nabla^2 \phi = \omega &\implies \text{控制体积分} \implies \text{面通量 } H_f, \\ H_f &\implies \text{隐式正交项} + \text{显式非正交项} \implies A\phi = b, \\ A\phi = b &\implies \text{fvm::laplacian(phi)}. \end{aligned}$$

其中每个内部面通量只计算一次，按 owner 加、neighbour 减进行守恒装配；OpenFOAM 代码中的每个空间格式均在前面的离散公式中有明确对应。

2 统一记号

全文采用此前两题已经统一的 owner-neighbour 记号, 不使用含义不清的相邻单元记号 $d(f)$, 也不以 A_f 表示面面积。

记号	含义
c, Ω_c, V_c	当前控制体、控制体区域及其体积
$\mathcal{F}_c^{\text{int}}$	与控制体 c 相接的内部面集合
$\mathcal{F}_c^{\text{bnd}}$	控制体 c 上的边界面集合
O_f, N_f	内部面 f 的 owner 单元和 neighbour 单元
$\mathbf{x}_{O_f}, \mathbf{x}_{N_f}$	owner 与 neighbour 的单元中心
$\mathbf{d}_f = \mathbf{x}_{N_f} - \mathbf{x}_{O_f}$	从 O_f 指向 N_f 的中心连线向量
$d_f = \ \mathbf{d}_f\ $	两单元中心距离
$\mathbf{n}_f$	从 O_f 指向 N_f 的全局单位法向量
$\mathbf{S}_f = S_f \mathbf{n}_f$	按 $O_f \rightarrow N_f$ 定向的全局有向面积向量
σ_{cf}	若 $c = O_f$ 则为 $+1$, 若 $c = N_f$ 则为 -1
$\mathbf{S}_{cf} = \sigma_{cf} \mathbf{S}_f$	相对于单元 c 的外向面积向量; 亦即 $\mathbf{S}_{cf} = S_f \mathbf{n}_{cf}$
H_f	按全局方向 $O_f \rightarrow N_f$ 定义的内部面扩散通量
Q_c	尚未除以体积的单元离散残差
$R_c = Q_c/V_c$	除以体积后的空间残差

这里 $|S_f|$ 是面的几何面积; 二维网格在有限体积意义下可理解为面长度乘挤出厚度。局部和全局面积向量之间始终满足

$$\mathbf{S}_{cf} = \sigma_{cf} \mathbf{S}_f = |S_f| \mathbf{n}_{cf}, \quad \sigma_{cf} = \begin{cases} +1, & c = O_f, \\ -1, & c = N_f. \end{cases} \quad (2)$$

3 从积分方程到有限体积空间离散

3.1 控制体积分与 Gauss 定理

在任意控制体 Ω_c 上积分方程 (1):

$$\int_{\Omega_c} \nabla \cdot (\nabla \phi) dV = \int_{\Omega_c} \omega dV. \quad (3)$$

应用 Gauss 散度定理, 得到精确的控制体平衡式

$$\oint_{\partial\Omega_c} \nabla \phi \cdot \mathbf{n}_c dS = \int_{\Omega_c} \omega dV. \quad (4)$$

将每个面上的面积分用面中心值近似, 并定义单元平均源项

$$\omega_c = \frac{1}{V_c} \int_{\Omega_c} \omega dV, \quad (5)$$

可得有限体积空间离散的基本形式

$$\sum_{f \in \mathcal{F}_c} (\nabla \phi)_f \cdot \mathbf{S}_{cf} = \omega_c V_c. \quad (6)$$

若 OpenFOAM 中的 ω_c 只是单元中心值, 则右端 $\omega_c V_c$ 相当于对源项采用单元中点求积。式 (6) 对应 Moukalled 等 [1, 第 8 章, 式 (8.4)] 给出的扩散项有限体积面通量和, 只是这里扩散系数等于 1, 且方程右端记为 ω 。

3.2 全局面通量及守恒装配

对每个内部面只计算一次全局定向通量

$$H_f = (\nabla \phi)_f \cdot \mathbf{S}_f. \quad (7)$$

它在 owner 单元方程中的外向贡献是 $+H_f$, 在 neighbour 单元方程中的外向贡献是 $-H_f$ 。因此, 可将式 (6) 写成

$$\sum_{f \in \mathcal{F}_c^{\text{int}}} \sigma_{cf} H_f + \sum_{b \in \mathcal{F}_c^{\text{bnd}}} H_{cb}^{\text{bnd}} = \omega_c V_c, \quad (8)$$

其中 $H_{cb}^{\text{bnd}} = (\nabla \phi)_b \cdot \mathbf{S}_{cb}$ 按控制体外法向定义。于是未归一化残差和归一化残差分别为

$$Q_c(\phi) := \sum_{f \in \mathcal{F}_c^{\text{int}}} \sigma_{cf} H_f + \sum_{b \in \mathcal{F}_c^{\text{bnd}}} H_{cb}^{\text{bnd}} - \omega_c V_c, \quad (9)$$

$$R_c(\phi) := \frac{Q_c(\phi)}{V_c}. \quad (10)$$

所求离散解满足

$$Q_c(\phi) = 0 \iff R_c(\phi) = 0, \quad \forall c. \quad (11)$$

注意: Q_c 是面通量和减去体积分源项, 尚未除以 V_c ; 只有 R_c 才是除以体积后的残差。

内部面的守恒装配规则为

$$Q_{O_f} += H_f, \quad Q_{N_f} -= H_f. \quad (12)$$

所以任一内部面在全域求和中严格成对抵消, 这是单元中心有限体积离散的局部守恒结构。这与 *OpenFOAM Technology Primer* 第 1.3.1 节的实现说明完全一致: OpenFOAM 的面积向量从 owner 指向 neighbour, 面贡献计算一次后加到 owner、从 neighbour 中减去 [2, 第 1.3.1 节, 第 28–29 页]。

4 正交网格上的面通量与矩阵

若 $\mathbf{S}_f \parallel \mathbf{d}_f$, 则面法向导数可用中心差分近似:

$$(\nabla \phi)_f \cdot \mathbf{n}_f \approx \frac{\phi_{N_f} - \phi_{O_f}}{d_f}. \quad (13)$$

因此

$$\boxed{H_f = D_f(\phi_{N_f} - \phi_{O_f}), \quad D_f = \frac{|S_f|}{d_f}.} \quad (14)$$

这就是 Moukalled 等 [1, 式 (8.63)] 在扩散系数为 1 时的正交网格面法向梯度公式。

原方程 $\nabla^2 \phi = \omega$ 直接离散会形成负对角矩阵。为了采用常用的正定形式进行线性求解，可将连续方程两边同时乘以 -1 ：

$$-\nabla^2 \phi = -\omega. \quad (15)$$

这不改变数学解。对每个内部面 f ，正定矩阵 A 的装配为

$$\begin{aligned} A_{O_f O_f} &+= D_f, & A_{O_f N_f} &-= D_f, \\ A_{N_f N_f} &+= D_f, & A_{N_f O_f} &-= D_f. \end{aligned} \quad (16)$$

内部面对应的局部块即

$$D_f \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad (17)$$

它是对称半正定的；施加足够的 Dirichlet 边界后，全局矩阵通常成为对称正定矩阵。

4.1 Dirichlet 边界

设边界面 b 上规定 $\phi_b = g_b$ ，单元中心到边界面中心的距离为 $d_{cb} = \|\mathbf{x}_b - \mathbf{x}_c\|$ 。在边界中心连线与面法向一致时，原方程的外向通量为

$$H_{cb}^{\text{bnd}} \approx D_b(g_b - \phi_c), \quad D_b = \frac{|S_b|}{d_{cb}}. \quad (18)$$

对正定形式 (15)，该边界面给出

$$A_{cc} += D_b, \quad b_c += D_b g_b, \quad b_c \text{ 的体源初值为 } -\omega_c V_c. \quad (19)$$

OpenFOAM v14 的普通非耦合 `fixedValue` 边界通过 `gradientInternalCoeffs =  $-\delta_b$` 和 `gradientBoundaryCoeffs =  $\delta_b g_b$` 形成上述通量系数 [7]。需要特别区分：Moukalled 等教材在非正交边界上还推导了理论上的交叉扩散修正 [1, 式 (8.81)–(8.83)]；而 OpenFOAM v14 对普通非耦合边界将非正交修正向量置零 [6]。因此，若需要保持边界附近的形式精度，应尽量使边界面中心连线与边界法向接近正交，或专门设计相应边界离散。

5 从课本非正交离散到 OpenFOAM 实现

5.1 教科书的几何分解

非正交网格中， $\mathbf{S}_f$ 与 $\mathbf{d}_f$ 不再平行。令

$$\mathbf{e}_f = \frac{\mathbf{d}_f}{d_f}. \quad (20)$$

Moukalled 等第 8.6 节和 *OpenFOAM Technology Primer* 式 (1.38)–(1.41) 都采用同一基本思想：将面积向量分解为

$$\mathbf{S}_f = \mathbf{E}_f + \mathbf{T}_f, \quad \mathbf{E}_f \parallel \mathbf{d}_f, \quad (21)$$

其中 $\mathbf{E}_f$ 是可由 ϕ_{O_f}, ϕ_{N_f} 隐式离散的部分， $\mathbf{T}_f$ 是需要当前梯度计算的非正交部分。OpenFOAM 的 `corrected Laplacian` 采用课本所述的 over-relaxed 分解：

$$\mathbf{E}_f = \frac{\mathbf{S}_f \cdot \mathbf{S}_f}{\mathbf{e}_f \cdot \mathbf{S}_f} \mathbf{e}_f, \quad \mathbf{T}_f = \mathbf{S}_f - \mathbf{E}_f. \quad (22)$$

从而

$$(\nabla \phi)_f \cdot \mathbf{S}_f = |\mathbf{E}_f| \frac{\phi_{N_f} - \phi_{O_f}}{d_f} + (\nabla \phi)_f \cdot \mathbf{T}_f. \quad (23)$$

第一项可由相邻单元值隐式线性化，第二项是非正交（交叉扩散）修正。Moukalled 等 [1, 式 (8.66)–(8.71), 第 8.6.5 节] 指出，交叉扩散项用当前梯度场显式计算，并以延迟修正方式加入代数方程右端；Primer 进一步说明，为使显式修正与新解相容，需要执行固定次数的非正交修正迭代 [2, 式 (1.38)–(1.41), 第 42–43 页]。

由式 (22) 可得课本离散中的隐式扩散系数

$$D_f := \frac{|\mathbf{E}_f|}{d_f} = \frac{|\mathbf{S}_f|^2}{\mathbf{S}_f \cdot \mathbf{d}_f}, \quad (24)$$

以及显式交叉扩散通量

$$C_f := (\nabla \phi)_f^{\text{lin}} \cdot \mathbf{T}_f. \quad (25)$$

于是内部面的全局通量首先由课本得到

$$\boxed{H_f = D_f(\phi_{N_f} - \phi_{O_f}) + C_f.} \quad (26)$$

这一步是后续矩阵和 OpenFOAM 程序的共同出发点。

单元梯度可由 Green–Gauss 公式近似：

$$(\nabla \phi)_c \approx \frac{1}{V_c} \sum_{f \in \mathcal{F}_c} \phi_f \mathbf{S}_{cf}, \quad (27)$$

其中内部面 ϕ_f 可由 ϕ_{O_f} 与 ϕ_{N_f} 线性插值。式 (27) 对应教材式 (8.72)–(8.76) [1]。

5.2 课本公式与 OpenFOAM v14 几何量的逐项对应

OpenFOAM v14 的 `correctedSnGrad` 用单位法向量形式实现同一分解：

$$\delta_f^\perp = \frac{1}{\max(\mathbf{n}_f \cdot \mathbf{d}_f, 0.05d_f)}, \quad (28)$$

$$\mathbf{k}_f = \mathbf{n}_f - \delta_f^\perp \mathbf{d}_f, \quad (29)$$

$$(\nabla \phi)_f \cdot \mathbf{n}_f \approx \delta_f^\perp (\phi_{N_f} - \phi_{O_f}) + \mathbf{k}_f \cdot (\nabla \phi)_f^{\text{lin}}. \quad (30)$$

其中 $(\nabla \phi)_f^{\text{lin}}$ 是单元梯度线性插值到面上的值。式 (28) 和 (29) 分别对应 OpenFOAM v14 的 `nonOrthDeltaCoeffs` 与 `nonOrthCorrectionVectors` [6]；式 (30) 对应 `correctedSnGrad::fullGradCorrection` [5]。

乘以 $|S_f|$ 后, OpenFOAM 内部面全局通量为

$$H_f = |S_f| \left[\delta_f^\perp (\phi_{N_f} - \phi_{O_f}) + \mathbf{k}_f \cdot (\nabla \phi)_f^{\text{lin}} \right]. \quad (31)$$

忽略式 (28) 中仅用于极端网格保护的 $0.05d_f$ 下限时,

$$|S_f| \delta_f^\perp = \frac{|S_f|^2}{\mathbf{S}_f \cdot \mathbf{d}_f} = D_f, \quad |S_f| \mathbf{k}_f = \mathbf{S}_f - D_f \mathbf{d}_f = \mathbf{T}_f. \quad (32)$$

因此式 (31) 逐项退化为课本式 (26), 不是另一套独立离散。公式中 $0.05d_f$ 只是 OpenFOAM v14 为极端非正交网格设置的数值下限, 不是从连续方程推导得到的数学常数 [6]。

在正交网格上, $\mathbf{n}_f \parallel \mathbf{d}_f$, 故 $\delta_f^\perp = 1/d_f$ 、 $\mathbf{k}_f = \mathbf{0}$, 式 (31) 退化为式 (14)。

对正定方程 $-\nabla^2 \phi = -\omega$, D_f 形成隐式矩阵系数, C_f 作为显式非正交修正加入右端:

$$b_{O_f} += C_f, \quad b_{N_f} -= C_f. \quad (33)$$

由于 C_f 依赖当前 ϕ 的梯度, 非正交网格上通常需要若干次非正交修正迭代。教材第 8.10.2 节明确说明, `fvm::laplacian` 返回隐式矩阵, 同时将非正交项加入源项 [1, 第 8.10.2 节及代码清单 8.5–8.9]; OpenFOAM v14 源代码也显示 `gaussLaplacianScheme` 先组装未修正矩阵, 再把 `snGrad` 的显式修正加入矩阵源项 [4]。

6 最终空间离散形式

把上面的通量写回单元方程, 原方程 $\nabla^2 \phi = \omega$ 的统一有限体积形式为

$$\sum_{f \in \mathcal{F}_c^{\text{int}}} \sigma_{cf} [D_f (\phi_{N_f} - \phi_{O_f}) + C_f] + \sum_{b \in \mathcal{F}_c^{\text{bnd}}} H_{cb}^{\text{bnd}} = \omega_c V_c. \quad (34)$$

其中 D_f 与 C_f 分别由式 (24) 和 (25) 定义; 在 OpenFOAM v14 中, 它们由等价的式 (31) 实现。这样, 最终方程直接来自课本的 $\mathbf{E}_f + \mathbf{T}_f$ 分解, 同时又与实际代码完全对应。等价的残差形式为

$$R_c(\phi) = \frac{1}{V_c} \left[\sum_{f \in \mathcal{F}_c^{\text{int}}} \sigma_{cf} H_f + \sum_{b \in \mathcal{F}_c^{\text{bnd}}} H_{cb}^{\text{bnd}} - \omega_c V_c \right] = 0. \quad (35)$$

这就是第 4.1 问所要求的空间“半离散”方程。若网格正交, 只需令 $\delta_f^\perp = 1/d_f$ 、 $\mathbf{k}_f = \mathbf{0}$, 即可得到中心差分形式。

7 基于 OpenFOAM 空间离散的求解器

7.1 由离散方程直接得到的算法

在写 OpenFOAM 代码以前, 先把式 (34) 转成明确的求解步骤。这样代码不是经验拼接, 而是代数方程的直接实现。

1. 读入网格、 ϕ 、 ω 和边界条件，给定当前迭代值 $\phi^{(m)}$ 。
2. 用式 (27) 计算单元梯度，并线性插值得到 $(\nabla\phi)_f^{\text{lin},(m)}$ ，进而由式 (25) 计算显式修正 $C_f^{(m)}$ 。
3. 对正定形式 $-\nabla^2\phi = -\omega$ ，先置 $b_c = -\omega_c V_c$ 。对每个内部面按式 (16) 把 D_f 装入矩阵，并按式 (33) 把 $C_f^{(m)}$ 装入右端。
4. 对 $\phi_b = g_b$ 的边界按式 (19) 加入对角系数和边界源项，得到

$$A\phi^{(m+1)} = b(\phi^{(m)}). \quad (36)$$

5. 求解线性系统，更新边界场；若网格非正交，则用新解重新计算梯度和 C_f ，执行规定次数的非正交修正。最后用原方程残差 (35) 检查是否闭合。

这正是课本的 deferred correction: D_f 隐式进入 A , C_f 用当前迭代值显式进入 b [1, 第 8.6.5、8.10.2 节]。Primer 第 1.3.2 节给出相同的隐式/显式拆分和非正交迭代，第 9.3 节演示如何把一个新模型方程写成 OpenFOAM 的 `fvm` 运算符 [2, 第 1.3.2、9.3 节]。

7.2 核心 C++ 程序

以下程序面向 OpenFOAM Foundation v14。为了得到正对角的对称系统，代码求解与原方程等价的 $-\nabla^2\phi = -\omega$ 。`simpleControl` 在这里仅负责稳态收敛控制和非正交修正计数，不引入任何新的物理方程；空间离散方法由算例的 `system/fvSchemes` 在运行时选择。

Listing 1: poissonFoam.C

```

1  #include "fvCFD.H"
2  #include "simpleControl.H"
3
4  int main(int argc, char *argv[])
5  {
6      #include "setRootCase.H"
7      #include "createTime.H"
8      #include "createMesh.H"
9
10     simpleControl simple(mesh);
11
12     volScalarField phi
13     (
14         IOobject
15         (
16             "phi",
17             runTime.name(),
18             mesh,
19             IOobject::MUST_READ,
20             IOobject::AUTO_WRITE
21         ),
22         mesh
23     );
24

```

```

25     volScalarField omega
26     (
27         IOobject
28         (
29             "omega",
30             runTime.name(),
31             mesh,
32             IOobject::MUST_READ,
33             IOobject::NO_WRITE
34         ),
35         mesh
36     );
37
38     Info<< "Solving laplacian(phi) = omega\n" << endl;
39
40     while (simple.loop(runTime))
41     {
42         while (simple.correctNonOrthogonal())
43         {
44             // Equivalent positive-definite form:
45             //      -laplacian(phi) = -omega
46             fvScalarMatrix phiEqn
47             (
48                 -fvm::laplacian(phi) == -omega
49             );
50
51             phiEqn.solve();
52         }
53
54         runTime.write();
55     }
56
57     Info<< "End\n" << endl;
58     return 0;
59 }

```

若只写与原方程同号的最简表达，也可以使用

```

1 fvScalarMatrix phiEqn(fvm::laplacian(phi) == omega);
2 phiEqn.solve();

```

两者解完全相同；前一种写法更直观地对应正定 Poisson 矩阵。OpenFOAM 中 `fvc::laplacian(phi)` 返回已显式计算的单元场，而 `fvm::laplacian(phi)` 返回可组装和求解的 `fvMatrix`，因此本题应采用后者 [1, 第 8.10.2 节]。这种“模型方程直接写成 `fvm` 表达式”的程序结构也与 Primer 第 9.3.4 节代码清单中的做法一致 [2, 第 9.3.4 节, 第 288–291 页]。

7.3 编译文件

Listing 2: Make/files

```
1 poissonFoam.C
2
3 EXE = $(FOAM_USER_APPBIN)/poissonFoam
```

Listing 3: Make/options

```
1 EXE_INC = \
2     -I$(LIB_SRC)/finiteVolume/lnInclude \
3     -I$(LIB_SRC)/meshTools/lnInclude
4
5 EXE_LIBS = \
6     -lfiniteVolume \
7     -lmeshTools
```

在求解器目录中执行 `wmake` 即可编译。

7.4 空间格式: system/fvSchemes

Listing 4: 本题所需的 fvSchemes 核心设置

```
1 gradSchemes
2 {
3     default      Gauss linear;
4     grad(phi)    Gauss linear;
5 }
6
7 laplacianSchemes
8 {
9     default      none;
10    laplacian(phi) Gauss linear corrected;
11 }
12
13 interpolationSchemes
14 {
15     default      linear;
16 }
17
18 snGradSchemes
19 {
20     default      corrected;
21 }
```

这组设置逐项含义如下:

1. **Gauss**: 使用 Gauss 定理把单元体积分转成面通量和; OpenFOAM 用户指南将其作为 Laplacian 离散的 Gauss 形式 [3]。
2. **linear**: 面上所需的量采用相邻单元中心值线性插值。
3. **corrected**: 使用式 (30) 的显式非正交修正; 其实现由 `correctedSnGrad` 源代码直接给出 [5]。

在完全正交网格上可把 `corrected` 换成 `orthogonal`; 但对于一般非结构网格, 直接使用 `orthogonal` 会忽略交叉扩散项。

7.5 线性求解与非正交修正: `system/fvSolution`

对乘以 -1 后的对称正定系统, 可采用 PCG 与 DIC 预条件。以下参数是一组可复现实验的选择, 并非原题唯一规定的数值:

Listing 5: `fvSolution` 核心设置

```

1 solvers
2 {
3     phi
4     {
5         solver          PCG;
6         preconditioner  DIC;
7         tolerance       1e-10;
8         relTol          0;
9     }
10 }
11
12 SIMPLE
13 {
14     nNonOrthogonalCorrectors 2;
15
16     residualControl
17     {
18         phi          1e-10;
19     }
20 }
```

对接近正交的网格, `nNonOrthogonalCorrectors` 可设为 0; 网格非正交性明显时可使用 1-2 次修正, 并通过残差及网格敏感性检查判断是否充分。OpenFOAM v14 的非正交控制类从算法字典读取该参数并执行修正循环 [8]。

7.6 场文件及边界条件

若 ϕ 无量纲, 则 ω 的量纲为 L^{-2} 。一个边界值场骨架为

Listing 6: `0/phi` 的边界骨架

```

1 dimensions    [0 0 0 0 0 0 0];
```

```
2 internalField    uniform 0;
3
4 boundaryField
5 {
6     left
7     {
8         type      fixedValue;
9         value      uniform 0; // 用题目给定的 g 替换
10    }
11
12    right
13    {
14        type      fixedValue;
15        value      uniform 0;
16    }
17
18    frontAndBack
19    {
20        type      empty;      // 二维挤出网格
21    }
22 }
```

源项场骨架为

Listing 7: Ω/ω 的量纲与内部场

```
1 dimensions      [0 -2 0 0 0 0 0];
2 internalField    uniform 0; // 替换为题目给定的 omega(x,y)
3
4 boundaryField
5 {
6     left
7     {
8         type      zeroGradient;
9     }
10
11    right
12    {
13        type      zeroGradient;
14    }
15
16    frontAndBack
17    {
18        type      empty;
19    }
20 }
```

ω 在本题中只作为单元体源使用，上述非空边界上的 `zeroGradient` 只是使场文件定义完整，不会替代 ϕ 的边界条件。若 ω 是坐标函数，可以在求解器中由网格单元中心 `mesh.C()` 计算，也可以先用 `setExprFields` 生成场。第 4.1 问只给出了抽象的 ω ，所以这里不擅自添加具体函数。

为了使 `simple.loop(runTime)` 有明确的稳态迭代上限，还需要在 `system/controlDict` 中给出运行控制。例如：

Listing 8: controlDict 的核心运行控制

```
1 application    poissonFoam;
2 startFrom      startTime;
3 startTime      0;
4 stopAt         endTime;
5 endTime        500;
6 deltaT         1;
7 writeControl    timeStep;
8 writeInterval  500;
```

这里的“时间步”只是稳态外迭代计数；方程中没有增加时间导数。若 `residualControl` 提前满足，`simpleControl` 会在到达 `endTime` 前结束。

8 公式、矩阵、配置和代码的闭环对应

下面的对应关系说明整个实现只使用了一套离散逻辑。

数学步骤	离散含义	OpenFOAM 实现
式 (4)	体积分转为控制体边界面通量和	Gauss Laplacian
$S_f = S_f n_f$ 与式 (12)	全面积向量从 O_f 指向 N_f ，面通量计算一次	owner()、neighbour() 及 ldu 矩阵装配
式 (14)	相邻单元中心差分形成隐式系数	fvm::laplacian(phi) 的矩阵部分
式 (27)	单元梯度及面梯度插值	grad(phi) Gauss linear
式 (21)–(26)	D_f 隐式、 C_f 显式的 deferred correction	Gauss linear corrected
式 (33)	非正交项加入右端并重新计算	correctNonOrthogonal()
式 (19)	已知边界值进入对角线和右端	fixedValue
式 (35)	原方程的单元归一化残差	phiEqn.solve() 的收敛控制

因此，代码中的 `fvm::laplacian(phi)` 并未跳过有限体积推导，而是把式 (16)、(33) 及边界装配封装为 `fvScalarMatrix`。`fvSchemes` 负责选择推导中尚未唯一确定的插值与非正交处理，`fvSolution` 只负责求解已经形成的代数系统。Primer 第 3.3 节也明确区分了这两类配置 [2, 第 3.3 节, 第 117–131 页]。

9 结论

第 4.1 问的核心答案是式 (34)或等价残差式 (35)。在 OpenFOAM v14 中,

```
fvm::laplacian(phi) + Gauss linear corrected
```

正好实现 “Gauss 面通量和 + 线性面插值 + 显式非正交修正”。内部面采用 $O_f \rightarrow N_f$ 的单一全局方向, 只计算一次 H_f , 然后 owner 加、neighbour 减; 残差记号始终为未归一化的 Q_c 与归一化的 $R_c = Q_c/V_c$ 。

参考文献

- [1] F. Moukalled, L. Mangani, and M. Darwish, *The Finite Volume Method in Computational Fluid Dynamics: An Advanced Introduction with OpenFOAM and Matlab*, Springer, 2016, DOI: [10.1007/978-3-319-16874-6](https://doi.org/10.1007/978-3-319-16874-6). 本解答主要使用第 8 章: 式 (8.4)、(8.63)–(8.77)、(8.81)–(8.83), 以及第 8.10.2 节 (第 211–265 页)。
- [2] T. Marić, J. Höpken, and K. G. Mooney, *The OpenFOAM Technology Primer*, Version OpenFOAM-v2012. 本解答使用第 1.3.1–1.3.2 节 (owner–neighbour、Laplacian 与非正交修正), 第 3.3 节 (fvSchemes/fvSolution), 第 9.3 节 (实现新 PDE) 及第 10.1 节 (边界条件)。此项为用户上传课本。
- [3] OpenFOAM Foundation, *OpenFOAM User Guide: Numerical schemes*, <https://doc.cfd.direct/openfoam/user-guide/fvschemes>, 访问日期: 2026-08-27。
- [4] OpenFOAM Foundation, *OpenFOAM v14 source: fvmLaplacian.C*, https://cpp.openfoam.org/v14/fvmLaplacian_8C_source.html, 访问日期: 2026-08-27。
- [5] OpenFOAM Foundation, *OpenFOAM v14 source: correctedSnGrad.C*, https://cpp.openfoam.org/v14/correctedSnGrad_8C_source.html, 访问日期: 2026-08-27。
- [6] OpenFOAM Foundation, *OpenFOAM v14 source: surfaceInterpolation.C*, https://cpp.openfoam.org/v14/surfaceInterpolation_8C_source.html, 访问日期: 2026-08-27。
- [7] OpenFOAM Foundation, *OpenFOAM v14 source: fixedValueFvPatchField.C*, https://cpp.openfoam.org/v14/fixedValueFvPatchField_8C_source.html, 访问日期: 2026-08-27。
- [8] OpenFOAM Foundation, *OpenFOAM v14 source: nonOrthogonalSolutionControl.C*, https://cpp.openfoam.org/v14/nonOrthogonalSolutionControl_8C_source.html, 访问日期: 2026-08-27。